

Solutions Homework 3 Spring 2026

2026-03-23

Problem 1

Here is the function. Comments explain the code.

```
#function to do Loess regression
#Input
#dat=data set with the predictor values in the first column and response is the second column
#lambda=bandwidth parameter
#numx=number of points at which to evaluate the model

MyLoess<-function(dat,lambda,numx)
{ #a function to implement Nadaraya-Watson kernel-weighted average
  #estimates the mean function at point x0

  x<-dat[,1]
  y<-dat[,2]
  n<-length(y)

  xsd<-sd(x) #sd of x points

  xpts<-seq(from=min(x),to=max(x),length.out=numx) #xpts at which function will be estimated

  locreg<-function(x0,x,y,xsd) #function fit a line at the point x0, with gaussian weights
  { weightvals<-dnorm(abs(x-x0),mean=0,sd=lambda*xsd)
    ab<-lm(y~x,weight=weightvals)$coef
    return(ab[[1]]+ab[[2]]*x0)
  }

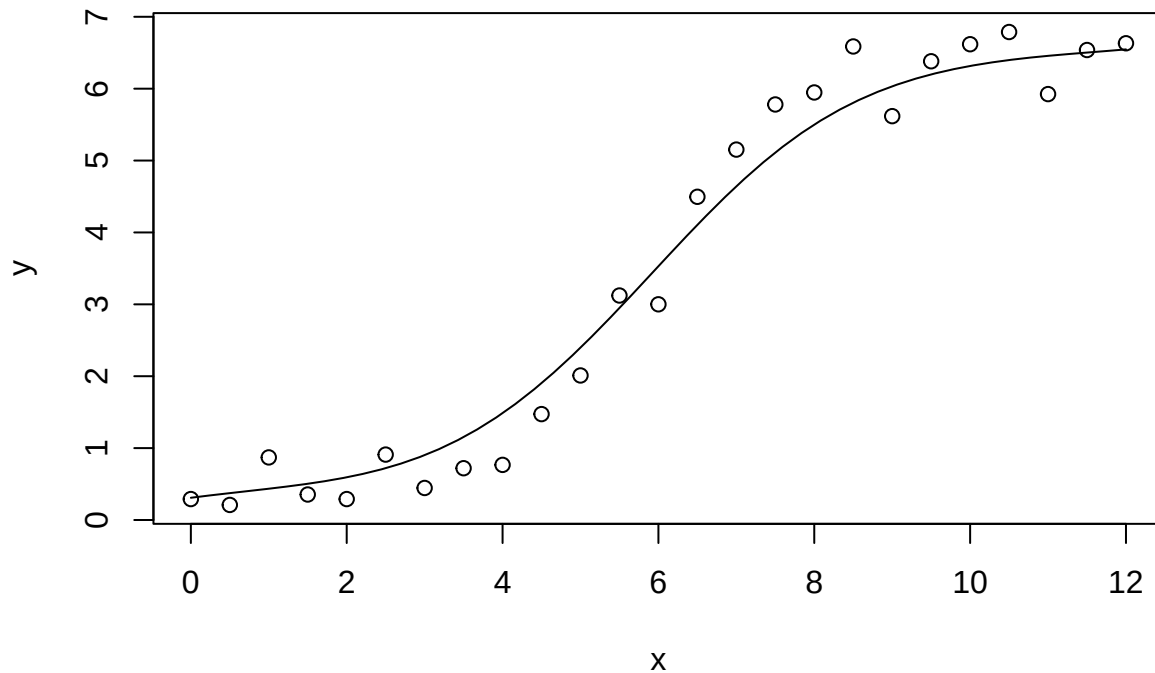
  yvals<-NULL
  for(x0 in xpts) yvals<-c(yvals,locreg(x0,x,y,xsd)) #find the fitted values

  out<-cbind(xpts,yvals)
  plot(dat,xlab="x", ylab="y")
  lines(out)

  return(out)
}
```

Example 1: Fit to the functional response data:

```
#example 1: Fit to function response data:
par(mfrow=c(1,1))
dat1<-read.table("../Data/FuncResp.txt")
#MyLoess(dat2,0.4,50)
MyLoess(dat1,0.4,50)
```

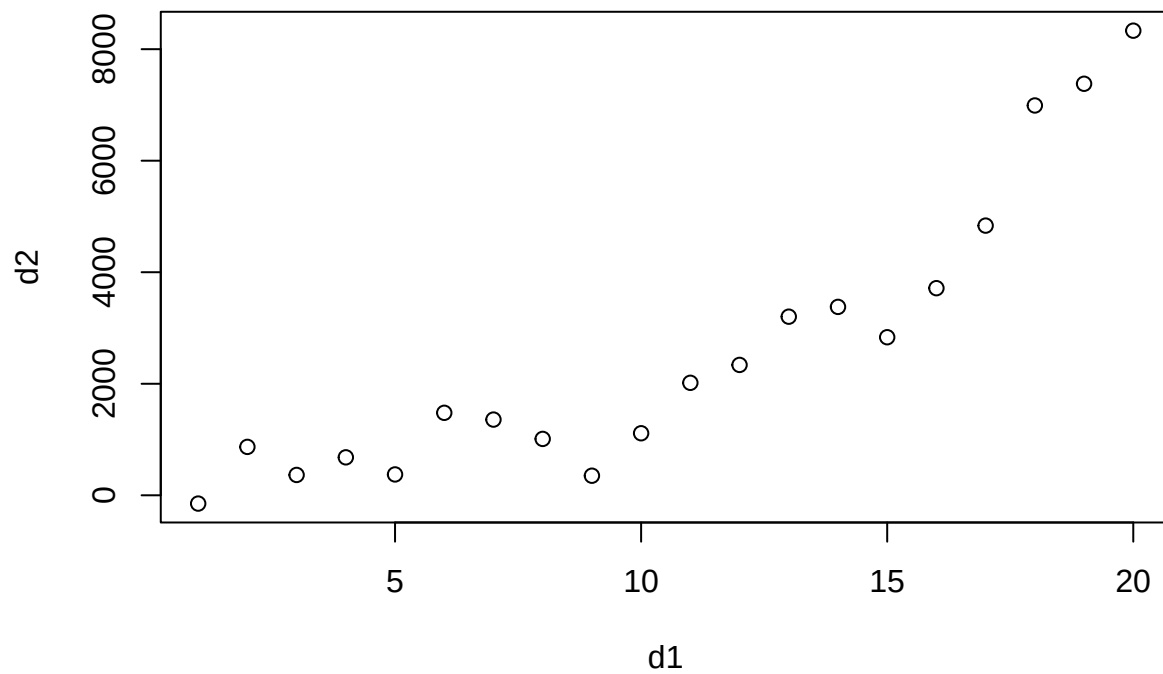


```
##          xpts      yvals
## [1,] 0.0000000 0.3093387
## [2,] 0.2448980 0.3418094
## [3,] 0.4897959 0.3725214
## [4,] 0.7346939 0.4024073
## [5,] 0.9795918 0.4325929
## [6,] 1.2244898 0.4643940
## [7,] 1.4693878 0.4993094
## [8,] 1.7142857 0.5390087
## [9,] 1.9591837 0.5853124
## [10,] 2.2040816 0.6401617
## [11,] 2.4489796 0.7055755
## [12,] 2.6938776 0.7835942
## [13,] 2.9387755 0.8762079
## [14,] 3.1836735 0.9852717
## [15,] 3.4285714 1.1124084
## [16,] 3.6734694 1.2589031
## [17,] 3.9183673 1.4255980
## [18,] 4.1632653 1.6127939
## [19,] 4.4081633 1.8201716
## [20,] 4.6530612 2.0467404
## [21,] 4.8979592 2.2908217
## [22,] 5.1428571 2.5500710
```

```
## [23,] 5.3877551 2.8215367
## [24,] 5.6326531 3.1017532
## [25,] 5.8775510 3.3868644
## [26,] 6.1224490 3.6727736
## [27,] 6.3673469 3.9553142
## [28,] 6.6122449 4.2304336
## [29,] 6.8571429 4.4943817
## [30,] 7.1020408 4.7438871
## [31,] 7.3469388 4.9763067
## [32,] 7.5918367 5.1897304
## [33,] 7.8367347 5.3830281
## [34,] 8.0816327 5.5558352
## [35,] 8.3265306 5.7084781
## [36,] 8.5714286 5.8418545
## [37,] 8.8163265 5.9572875
## [38,] 9.0612245 6.0563725
## [39,] 9.3061224 6.1408383
## [40,] 9.5510204 6.2124328
## [41,] 9.7959184 6.2728427
## [42,] 10.0408163 6.3236492
## [43,] 10.2857143 6.3663151
## [44,] 10.5306122 6.4021990
## [45,] 10.7755102 6.4325858
## [46,] 11.0204082 6.4587254
## [47,] 11.2653061 6.4818672
## [48,] 11.5102041 6.5032843
## [49,] 11.7551020 6.5242814
## [50,] 12.0000000 6.5461841
```

Example 2: Fit to cubic data:

```
#Make some data with y having a cubic dependence on x plus noise:
d1<-seq(from=1,to=20,by=1)
d2<-d1^3-d1+rnorm(length(d1),sd=500)
plot(d1,d2)
```



```
dat2<-cbind(d1,d2)

#Run my function:
MyLoess(dat2,0.5,50)
```

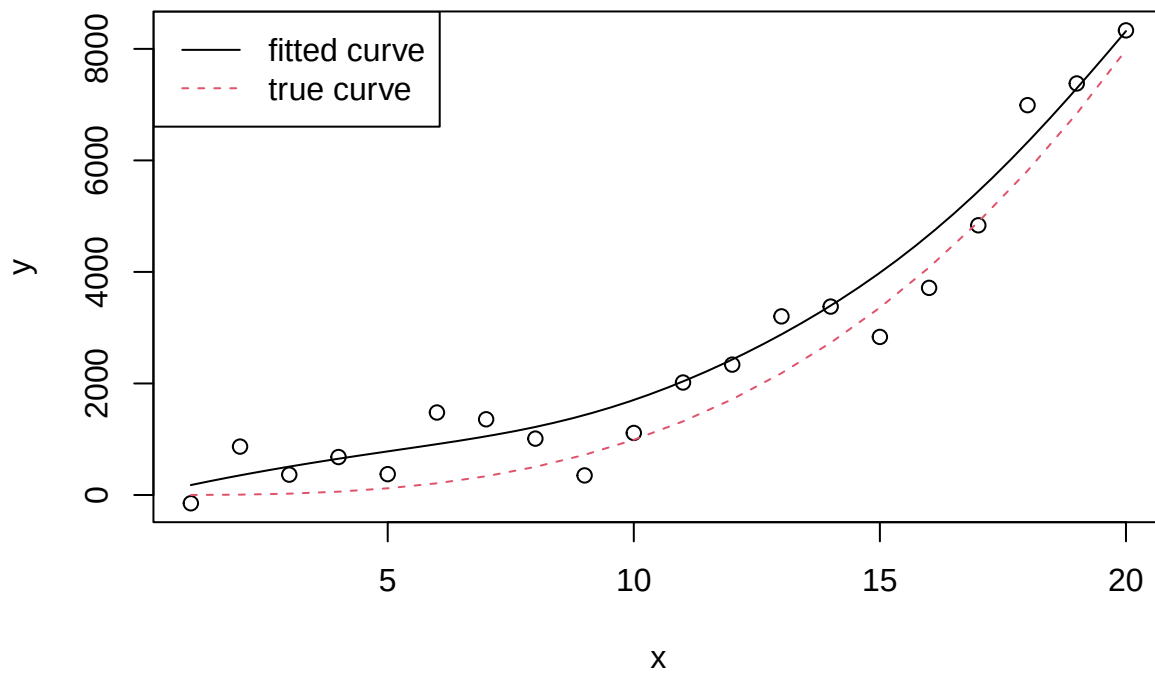
```
##          xpts      yvals
## [1,]  1.000000  178.1552
## [2,]  1.387755  248.3299
## [3,]  1.775510  315.2295
## [4,]  2.163265  379.2162
## [5,]  2.551020  440.5798
## [6,]  2.938776  499.5552
## [7,]  3.326531  556.3462
## [8,]  3.714286  611.1545
## [9,]  4.102041  664.2114
## [10,] 4.489796  715.8098
## [11,] 4.877551  766.3329
## [12,] 5.265306  816.2768
## [13,] 5.653061  866.2646
## [14,] 6.040816  917.0498
## [15,] 6.428571  969.5069
## [16,] 6.816327 1024.6114
## [17,] 7.204082 1083.4069
## [18,] 7.591837 1146.9644
## [19,] 7.979592 1216.3339
## [20,] 8.367347 1292.4944
## [21,] 8.755102 1376.3059
## [22,] 9.142857 1468.4693
## [23,] 9.530612 1569.4998
## [24,] 9.918367 1679.7169
## [25,] 10.306122 1799.2560
## [26,] 10.693878 1928.0989
```

```
## [27,] 11.081633 2066.1230
## [28,] 11.469388 2213.1614
## [29,] 11.857143 2369.0677
## [30,] 12.244898 2533.7776
## [31,] 12.632653 2707.3589
## [32,] 13.020408 2890.0453
## [33,] 13.408163 3082.2505
## [34,] 13.795918 3284.5611
## [35,] 14.183673 3497.7107
## [36,] 14.571429 3722.5373
## [37,] 14.959184 3959.9268
## [38,] 15.346939 4210.7493
## [39,] 15.734694 4475.7903
## [40,] 16.122449 4755.6831
## [41,] 16.510204 5050.8473
## [42,] 16.897959 5361.4366
## [43,] 17.285714 5687.3015
## [44,] 17.673469 6027.9684
## [45,] 18.061224 6382.6369
## [46,] 18.448980 6750.1953
## [47,] 18.836735 7129.2531
## [48,] 19.224490 7518.1876
## [49,] 19.612245 7915.2009
## [50,] 20.000000 8318.3834
```

#Add in the true curve:

```
lines(d1,d1^3-d1,col=2,lty=2)
```

```
legend("topleft",c("fitted curve","true curve"),lty=1:2,col=1:2)
```



The fit is usually quite good.

Example 3: Fit to cubic data with more noise

This is the same as example 2, but with more noise.

```
#Make some data with y having a cubic dependence on x plus noise:
d1<-seq(from=1,to=20,by=1)
d3<-d1^3-d1+rnorm(length(d1),sd=1000)

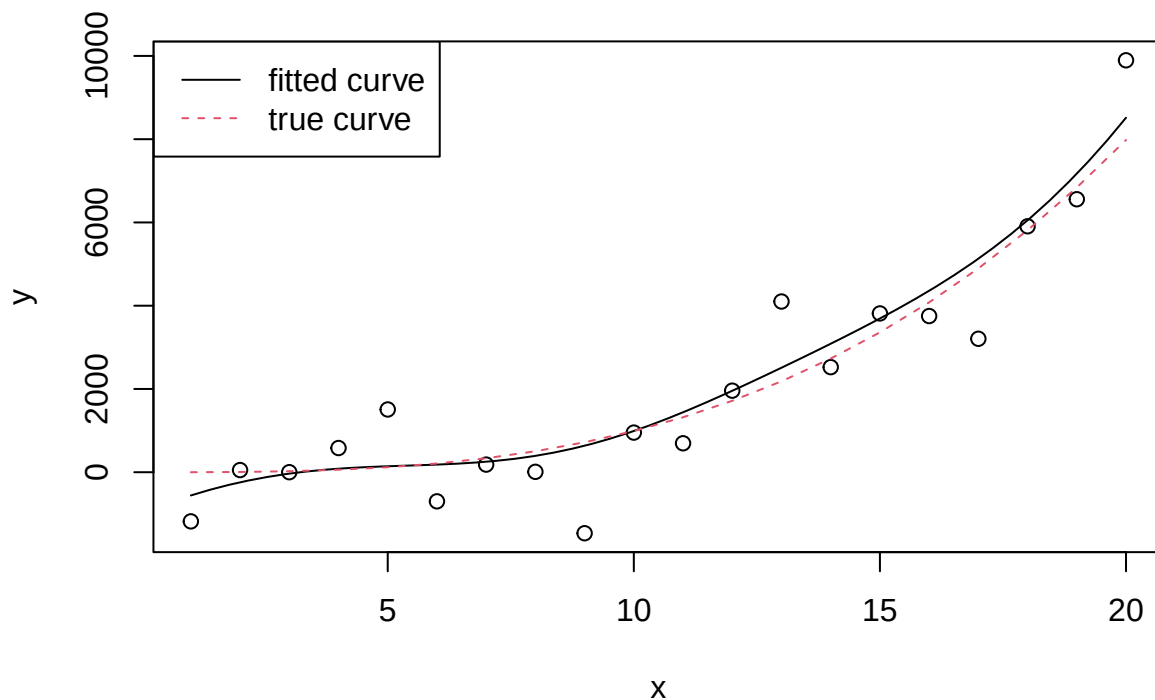
dat3<-cbind(d1,d3)

#Run my function:
MyLoess(dat3,0.5,50)
```

```
##          xpts      yvals
## [1,]  1.000000 -557.77676
## [2,]  1.387755 -421.94113
## [3,]  1.775510 -302.95941
## [4,]  2.163265 -200.54802
## [5,]  2.551020 -114.14948
## [6,]  2.938776  -42.90652
## [7,]  3.326531   14.35443
## [8,]  3.714286   59.12866
## [9,]  4.102041   93.22579
## [10,] 4.489796  118.74877
## [11,] 4.877551  138.05894
## [12,] 5.265306  153.72775
## [13,] 5.653061  168.47609
## [14,] 6.040816  185.10284
## [15,] 6.428571  206.40464
## [16,] 6.816327  235.08908
## [17,] 7.204082  273.68398
## [18,] 7.591837  324.44613
## [19,] 7.979592  389.27337
## [20,] 8.367347  469.62555
## [21,] 8.755102  566.46035
## [22,] 9.142857  680.19123
## [23,] 9.530612  810.67409
## [24,] 9.918367  957.22808
## [25,] 10.306122 1118.69300
## [26,] 10.693878 1293.52213
## [27,] 11.081633 1479.90489
## [28,] 11.469388 1675.91004
## [29,] 11.857143 1879.63764
## [30,] 12.244898 2089.36698
## [31,] 12.632653 2303.68881
## [32,] 13.020408 2521.61275
## [33,] 13.408163 2742.64427
## [34,] 13.795918 2966.82888
## [35,] 14.183673 3194.76462
## [36,] 14.571429 3427.58549
## [37,] 14.959184 3666.91992
## [38,] 15.346939 3914.82872
## [39,] 15.734694 4173.72699
## [40,] 16.122449 4446.29438
```

```
## [41,] 16.510204 4735.37817
## [42,] 16.897959 5043.89388
## [43,] 17.285714 5374.72766
## [44,] 17.673469 5730.64521
## [45,] 18.061224 6114.21081
## [46,] 18.448980 6527.71999
## [47,] 18.836735 6973.14780
## [48,] 19.224490 7452.11395
## [49,] 19.612245 7965.86491
## [50,] 20.000000 8515.27172
```

```
#Add in the true curve:
lines(d1,d1^3-d1,col=2,lty=2)
legend("topleft",c("fitted curve","true curve"),lty=1:2,col=1:2)
```



The fit still usually is good, sometimes with some problems at the ends.

Example 4

This example uses a sine curve as the underlying relationship. This is more complex than the previous examples

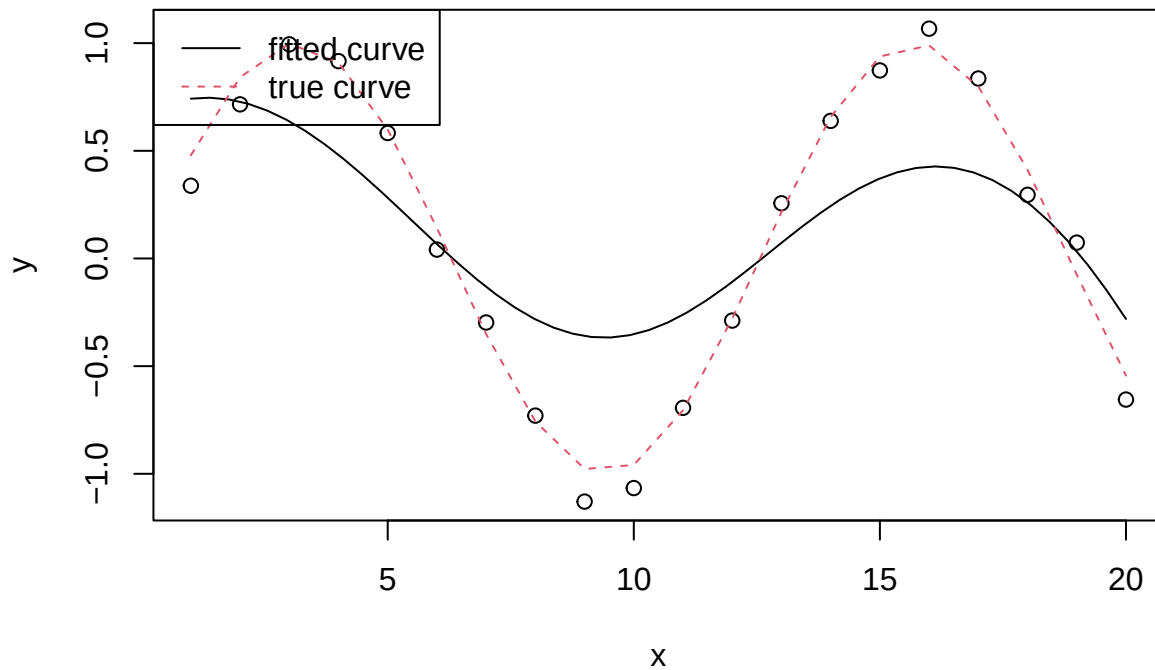
```
#low variance:
d1<-seq(from=1,to=20,by=1)
d4=sin(d1/2)+rnorm(length(d1),sd=0.1)
dat4=cbind(d1,d4)

MyLoess(dat4,0.5,50)
```

```
##          xpts          yvals
```

```
## [1,] 1.000000 0.742191149
## [2,] 1.387755 0.746390952
## [3,] 1.775510 0.738345844
## [4,] 2.163265 0.718348041
## [5,] 2.551020 0.686872639
## [6,] 2.938776 0.644585656
## [7,] 3.326531 0.592347779
## [8,] 3.714286 0.531213223
## [9,] 4.102041 0.462423280
## [10,] 4.489796 0.387394284
## [11,] 4.877551 0.307699912
## [12,] 5.265306 0.225047875
## [13,] 5.653061 0.141251196
## [14,] 6.040816 0.058194272
## [15,] 6.428571 -0.022206032
## [16,] 6.816327 -0.098044058
## [17,] 7.204082 -0.167473501
## [18,] 7.591837 -0.228759540
## [19,] 7.979592 -0.280333998
## [20,] 8.367347 -0.320851870
## [21,] 8.755102 -0.349246698
## [22,] 9.142857 -0.364781350
## [23,] 9.530612 -0.367090176
## [24,] 9.918367 -0.356208347
## [25,] 10.306122 -0.332584720
## [26,] 10.693878 -0.297075792
## [27,] 11.081633 -0.250920041
## [28,] 11.469388 -0.195693946
## [29,] 11.857143 -0.133252857
## [30,] 12.244898 -0.065661316
## [31,] 12.632653 0.004881803
## [32,] 13.020408 0.076118043
## [33,] 13.408163 0.145799392
## [34,] 13.795918 0.211750271
## [35,] 14.183673 0.271919551
## [36,] 14.571429 0.324421976
## [37,] 14.959184 0.367569373
## [38,] 15.346939 0.399892587
## [39,] 15.734694 0.420155260
## [40,] 16.122449 0.427360513
## [41,] 16.510204 0.420751377
## [42,] 16.897959 0.399805616
## [43,] 17.285714 0.364225506
## [44,] 17.673469 0.313923065
## [45,] 18.061224 0.249001355
## [46,] 18.448980 0.169732562
## [47,] 18.836735 0.076533687
## [48,] 19.224490 -0.030059222
## [49,] 19.612245 -0.149417339
## [50,] 20.000000 -0.280844887
```

```
lines(d1,sin(d1/2),col=2,lty=2)
legend("topleft",c("fitted curve","true curve"),lty=1:2,col=1:2)
```

I does not perform well with this bandwidth.

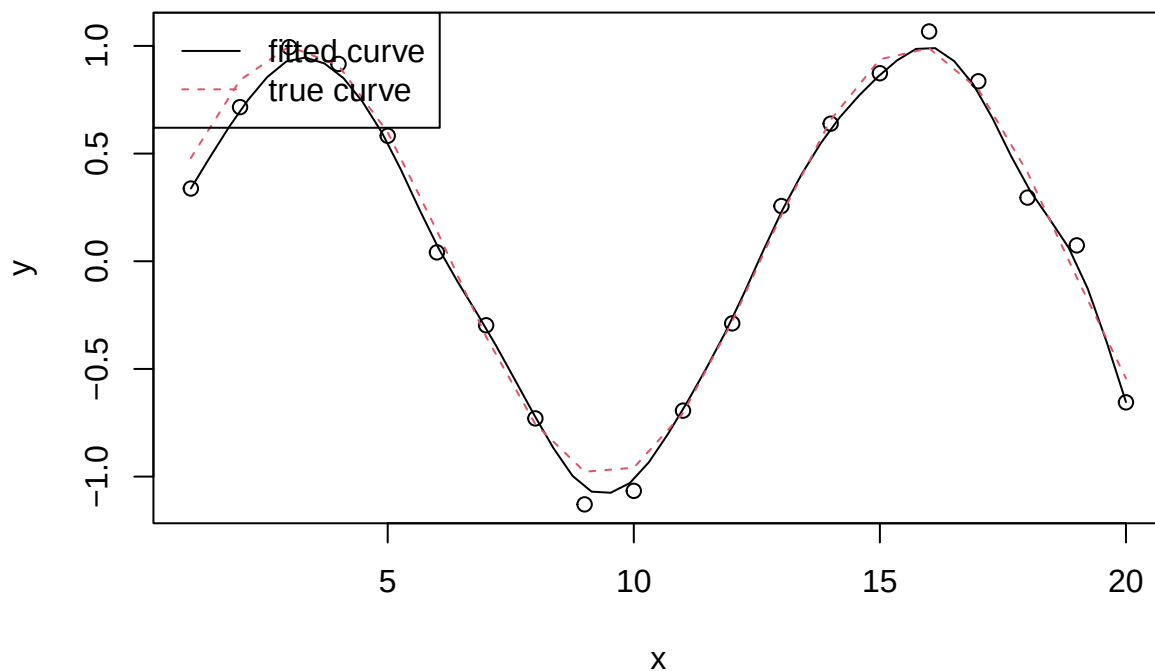
Low variance with smaller bandwidth:

```
MyLoess(dat4,0.1,50)
```

```
##          xpts          yvals
## [1,] 1.000000 0.33807364
## [2,] 1.387755 0.48313755
## [3,] 1.775510 0.62314561
## [4,] 2.163265 0.74957383
## [5,] 2.551020 0.85499269
## [6,] 2.938776 0.92766725
## [7,] 3.326531 0.94616985
## [8,] 3.714286 0.91858787
## [9,] 4.102041 0.85023000
## [10,] 4.489796 0.74005801
## [11,] 4.877551 0.59915549
## [12,] 5.265306 0.42655489
## [13,] 5.653061 0.23634797
## [14,] 6.040816 0.05644848
## [15,] 6.428571 -0.09835510
## [16,] 6.816327 -0.24244809
## [17,] 7.204082 -0.39321702
## [18,] 7.591837 -0.55358816
## [19,] 7.979592 -0.71508594
## [20,] 8.367347 -0.86781379
## [21,] 8.755102 -0.99682204
## [22,] 9.142857 -1.06998299
## [23,] 9.530612 -1.07499002
## [24,] 9.918367 -1.02990833
## [25,] 10.306122 -0.93349237
## [26,] 10.693878 -0.80224826
```

```
## [27,] 11.081633 -0.65553378
## [28,] 11.469388 -0.49911651
## [29,] 11.857143 -0.33169832
## [30,] 12.244898 -0.14612855
## [31,] 12.632653  0.05077993
## [32,] 13.020408  0.23881553
## [33,] 13.408163  0.40337162
## [34,] 13.795918  0.54711166
## [35,] 14.183673  0.66805935
## [36,] 14.571429  0.76827711
## [37,] 14.959184  0.85659171
## [38,] 15.346939  0.93249192
## [39,] 15.734694  0.98611139
## [40,] 16.122449  0.98957984
## [41,] 16.510204  0.92914758
## [42,] 16.897959  0.82093472
## [43,] 17.285714  0.66498360
## [44,] 17.673469  0.48522527
## [45,] 18.061224  0.32354596
## [46,] 18.448980  0.19554952
## [47,] 18.836735  0.06253191
## [48,] 19.224490 -0.12809311
## [49,] 19.612245 -0.37846206
## [50,] 20.000000 -0.65367607
```

```
lines(d1,sin(d1/2),col=2,lty=2)
legend("topleft",c("fitted curve","true curve"),lty=1:2,col=1:2)
```



Higher variance:

```
#low variance:
d1<-seq(from=1,to=20,by=1)
```

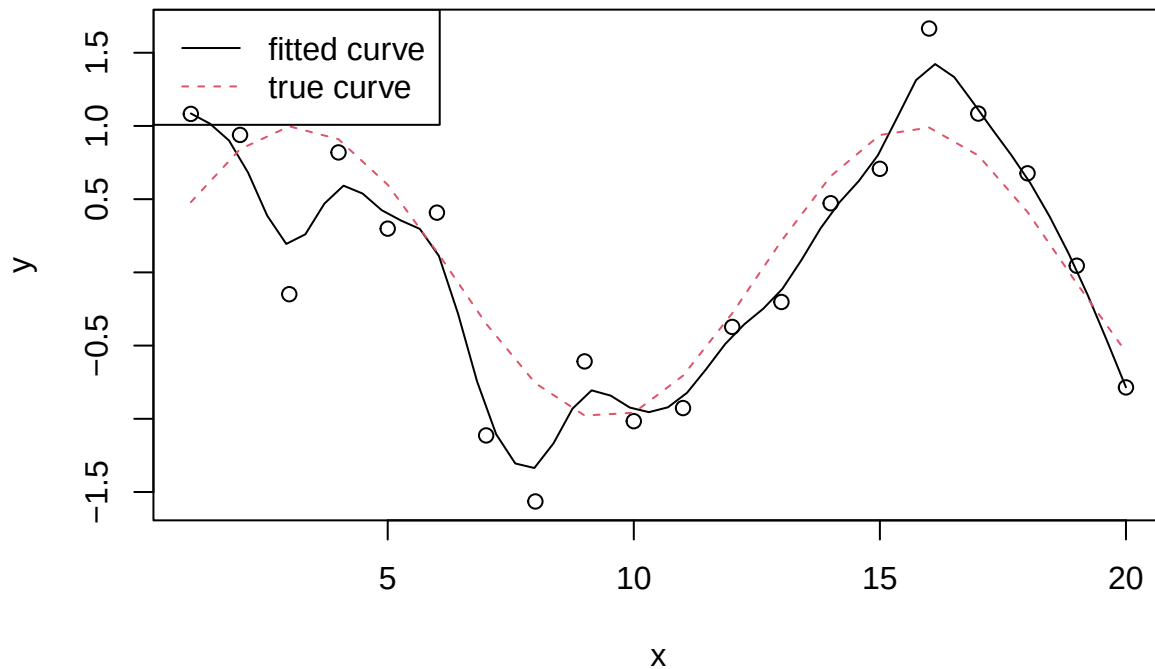
```
d4=sin(d1/2)+rnorm(length(d1),sd=0.5)
dat4=cbind(d1,d4)

MyLoess(dat4,0.1,50)
```

```
##           xpts          yvals
## [1,]  1.000000  1.08530635
## [2,]  1.387755  1.01577340
## [3,]  1.775510  0.90067457
## [4,]  2.163265  0.68096160
## [5,]  2.551020  0.38784226
## [6,]  2.938776  0.19421392
## [7,]  3.326531  0.26003517
## [8,]  3.714286  0.46985224
## [9,]  4.102041  0.59219913
## [10,] 4.489796  0.53838044
## [11,] 4.877551  0.42484903
## [12,] 5.265306  0.35477605
## [13,] 5.653061  0.29691965
## [14,] 6.040816  0.11158249
## [15,] 6.428571 -0.28210767
## [16,] 6.816327 -0.74547138
## [17,] 7.204082 -1.10593061
## [18,] 7.591837 -1.30511137
## [19,] 7.979592 -1.33600734
## [20,] 8.367347 -1.16806182
## [21,] 8.755102 -0.93025738
## [22,] 9.142857 -0.80562318
## [23,] 9.530612 -0.84176645
## [24,] 9.918367 -0.92323607
## [25,] 10.306122 -0.95475900
## [26,] 10.693878 -0.92128979
## [27,] 11.081633 -0.82249455
## [28,] 11.469388 -0.66130368
## [29,] 11.857143 -0.48920272
## [30,] 12.244898 -0.35455608
## [31,] 12.632653 -0.24724183
## [32,] 13.020408 -0.11176038
## [33,] 13.408163  0.08504283
## [34,] 13.795918  0.30125258
## [35,] 14.183673  0.48018886
## [36,] 14.571429  0.62396850
## [37,] 14.959184  0.79940208
## [38,] 15.346939  1.05451028
## [39,] 15.734694  1.31288919
## [40,] 16.122449  1.42284885
## [41,] 16.510204  1.33366302
## [42,] 16.897959  1.15816758
## [43,] 17.285714  0.97605336
## [44,] 17.673469  0.79978747
## [45,] 18.061224  0.60704042
## [46,] 18.448980  0.38137149
## [47,] 18.836735  0.12788629
```

```
## [48,] 19.224490 -0.15631655
## [49,] 19.612245 -0.46559588
## [50,] 20.000000 -0.78481247
```

```
lines(d1,sin(d1/2),col=2,lty=2)
legend("topleft",c("fitted curve","true curve"),lty=1:2,col=1:2)
```



shows a lot of effect of noise.

It

Problem 2

The goal is to produce a function that finds the optimal smoothing parameter for loess regression. We use the `loess` function from R. The smoothing parameter in `loess` is called `span`. Here is an outline of the procedure:

randomly assign data to k folds

loop j over folds

{ loop i over possible values of `span`

{ fit loess model to data with fold j left out using `span i`

use fitted model to predict response in fold j

calculate and store mse between actual and predicted response values for fold j

}

}

find the average MSE over folds for each value of `span`

Output: optimal `span` is value that produces lowest average MSE

Here is the code:

```
# dat is a data frame with the single predictor in column 1
# and the response variable in column 2
#smoothvec is the vector of values for the smoothing parameter that the function will search
```

#k specifies the number of folds in k-fold cross validation, and plotY is a TRUE/FALSE variable determining whether to plot the results

```
findspan<-function(dat,spanvec,k,plotY=TRUE)
{ n=nrow(dat)  #sample size
  folds=sample(1:k,n,replace= T)  #randomly assign data points to folds
  mse=matrix(nrow=k,ncol=length(spanvec))  #matrix to hold mse values

  for(j in 1:k)  #loop over folds
  { for(i in 1:length(spanvec))  #loop over spans to search
    { lfr=loess(dat[folds!=j,2]~dat[folds!=j,1],span=spanvec[i])  #loess model
      pred=predict(lfr,newdata=dat[folds==j,1])  #predicted value for held out fold
      mse[j,i]=mean((pred-dat[folds==j,2])^2,na.rm=T)  #mse between predicted and actual
    }
  }

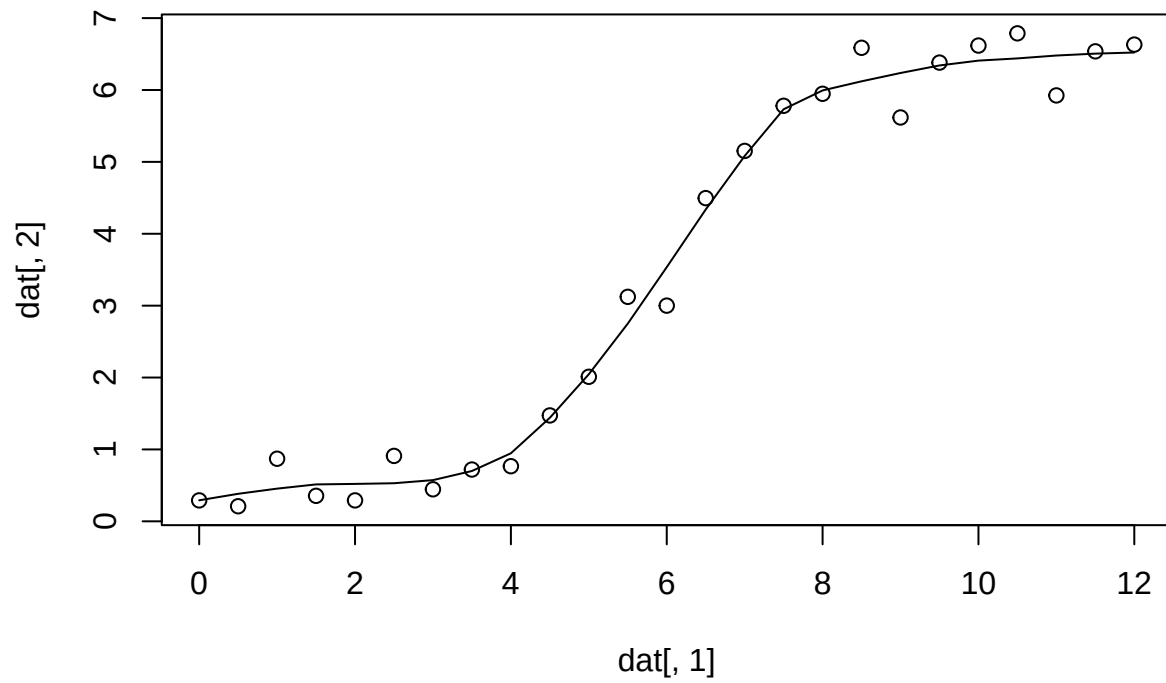
  #find the span that produces the lowest average mse over folds
  bestspan=spanvec[which.min(apply(mse,2,mean,na.rm=T))]

  if(plotY==TRUE)
  {
    plot(dat[,1],dat[,2])
    lfr2=loess(dat[,2]~dat[,1],span=bestspan)
    lines(lfr2$x,fitted(lfr2))
  }

  return(bestspan)
}
```

Here is a test with the functional response data:

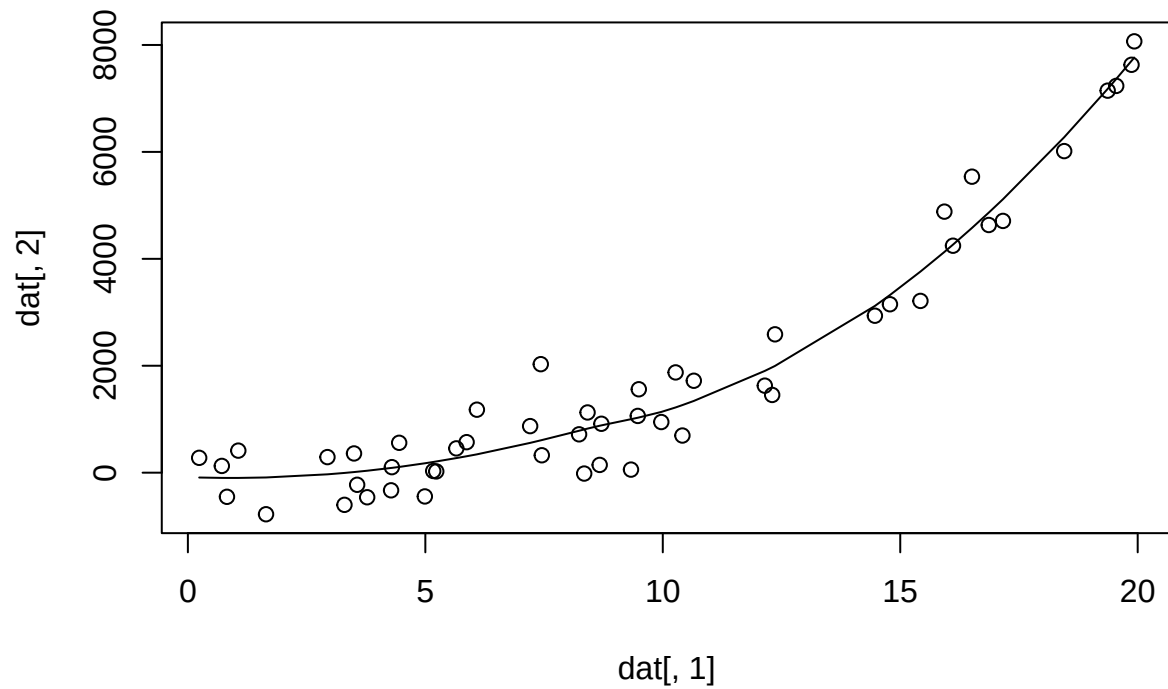
```
k=10
spanvec=seq(from=0.2,to=20,by=0.2)
dat2<-read.table("../Data/FuncResp.txt")
findspan(dat2,spanvec,10)
```



```
## [1] 0.4
```

Here is a test with simulated data:

```
n=50
set.seed(22)
dx1<-sort(runif(n,0,20))
dy1<-dx1^3-dx1+rnorm(length(dx1),sd=500)
#plot(dx1,dy1)
spanvec=seq(from=0.2,to=20,by=0.2)
dat1<-cbind(dx1,dy1)
dat1=data.frame(dat1)
names(dat1)=c("x","y")
findspan(dat1,spanvec,10)
```

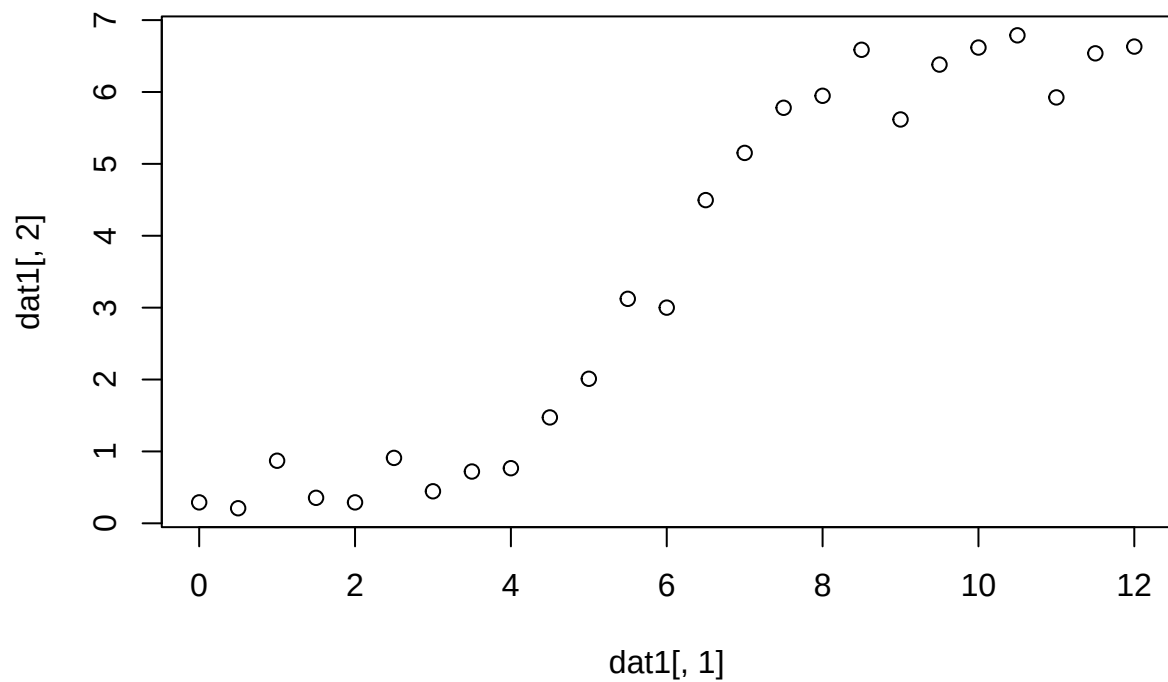


```
## [1] 0.8
```

Problem 3

I will use the functional response data to test with

```
dat1<-read.table("../Data/FuncResp.txt")
names(dat1)<-c("PreyDens","PreyEaten")
plot(dat1[,1],dat1[,2])
```



The function is below. Comments explain the code.

```
cubicspline<-function(dat,K)
{#This function fits natural cubic splines with K interior knots to the data in dat
#dat is a matrix where the first column is the predictor and the second column is the response.
# it calculates the basis matrix, fits a linear model to the y versus basis matrix data, and plots it.

  x<-dat[,1] #predictor
  y<-dat[,2] #response
  n<-length(x) #sample size
  basismat<-matrix(nrow=n,ncol=K) #will be the basis matrix

  knots<-quantile(x,probs=seq(from=0,to=1,length.out=K+2))[2:(K+1)]
    #k equally spaced interior knots + 2 boundary knots

  dk<-function(x,k,K) #dk is the d function for calculating the basis functions (see assignment sheet)
  {
    return (pmax((x-knots[k])^3,0)-pmax((x-knots[K])^3,0)/(knots[K]-knots[k]))
  }

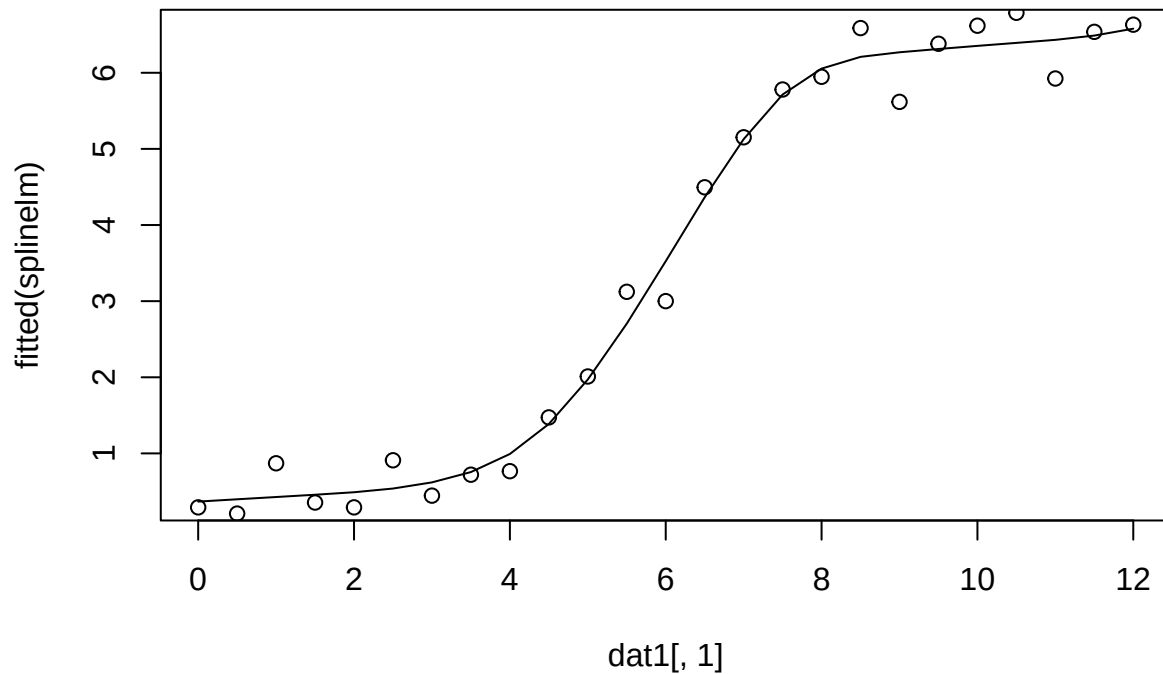
  basismat[,1]=1 #fill in basis matrix: N1
  basismat[,2]=x #N2
  for(j in 3:K) basismat[,j]=(dk(x,j-2,K)-dk(x,K-1,K)) #fill in basis matrix values: N3..NK

  splinelm<-lm(y~0+basismat) #fit linear model with basimat as the matrix of predictor values and n
  par(mfrow=c(1,1))
  plot(dat1[,1],fitted(splinelm),type="l") #plot the fitted spline
  points(dat1) #add in the data

  out<-list(basismat,splinelm) #this is the output of the function
  names(out)<-c("basis matrix","linear model")
  return(out)
}
```

Test:

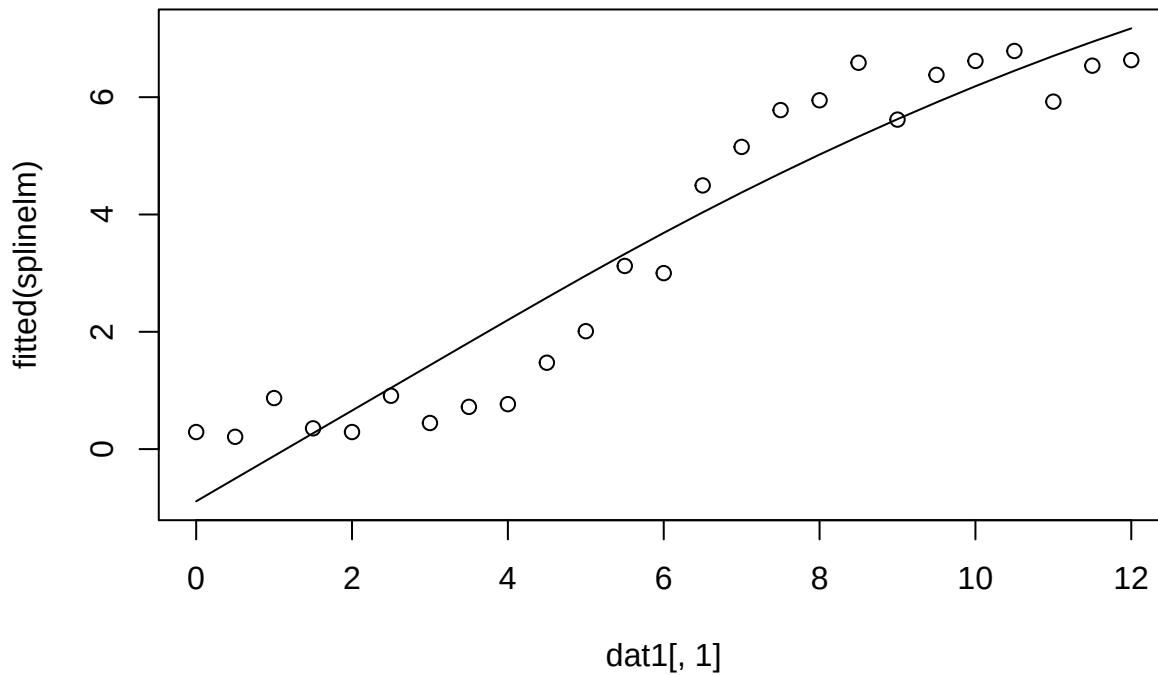
```
cubicspline(dat1,7)
```

```
## $'basis matrix'
##      [,1] [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,]  1  0.0   0.0000   0.0000   0.0000   0.0000   0.00000
## [2,]  1  0.5   0.0000   0.0000   0.0000   0.0000   0.00000
## [3,]  1  1.0   0.0000   0.0000   0.0000   0.0000   0.00000
## [4,]  1  1.5   0.0000   0.0000   0.0000   0.0000   0.00000
## [5,]  1  2.0   0.1250   0.0000   0.0000   0.0000   0.00000
## [6,]  1  2.5   1.0000   0.0000   0.0000   0.0000   0.00000
## [7,]  1  3.0   3.3750   0.0000   0.0000   0.0000   0.00000
## [8,]  1  3.5   8.0000   0.1250   0.0000   0.0000   0.00000
## [9,]  1  4.0  15.6250   1.0000   0.0000   0.0000   0.00000
## [10,] 1  4.5  27.0000   3.3750   0.0000   0.0000   0.00000
## [11,] 1  5.0  42.8750   8.0000   0.1250   0.0000   0.00000
## [12,] 1  5.5  64.0000  15.6250   1.0000   0.0000   0.00000
## [13,] 1  6.0  91.1250  27.0000   3.3750   0.0000   0.00000
## [14,] 1  6.5 125.0000  42.8750   8.0000   0.1250   0.00000
## [15,] 1  7.0 166.3750  64.0000  15.6250   1.0000   0.00000
## [16,] 1  7.5 216.0000  91.1250  27.0000   3.3750   0.00000
## [17,] 1  8.0 274.6250 125.0000  42.8750   8.0000   0.12500
## [18,] 1  8.5 343.0000 166.3750  64.0000  15.6250   1.00000
## [19,] 1  9.0 421.8750 216.0000  91.1250  27.0000   3.37500
## [20,] 1  9.5 511.8750 274.5000 124.8750  42.7500   7.87500
## [21,] 1 10.0 613.1250 342.0000 165.3750  63.0000  14.62500
## [22,] 1 10.5 725.6250 418.5000 212.6250  87.7500  23.62500
## [23,] 1 11.0 849.4444 504.0667 266.6875 117.0556  34.91667
## [24,] 1 11.5 984.9306 599.0333 327.8750 151.1944  48.70833
## [25,] 1 12.0 1132.5000 703.8000 396.5625 190.5000  65.25000
##
## $'linear model'
##
## Call:
## lm(formula = y ~ 0 + basismat)
```

```
##
## Coefficients:
## basismat1 basismat2 basismat3 basismat4 basismat5 basismat6 basismat7
## 0.36729 0.05986 0.02146 0.05047 -0.15400 -0.05288 0.25769
```

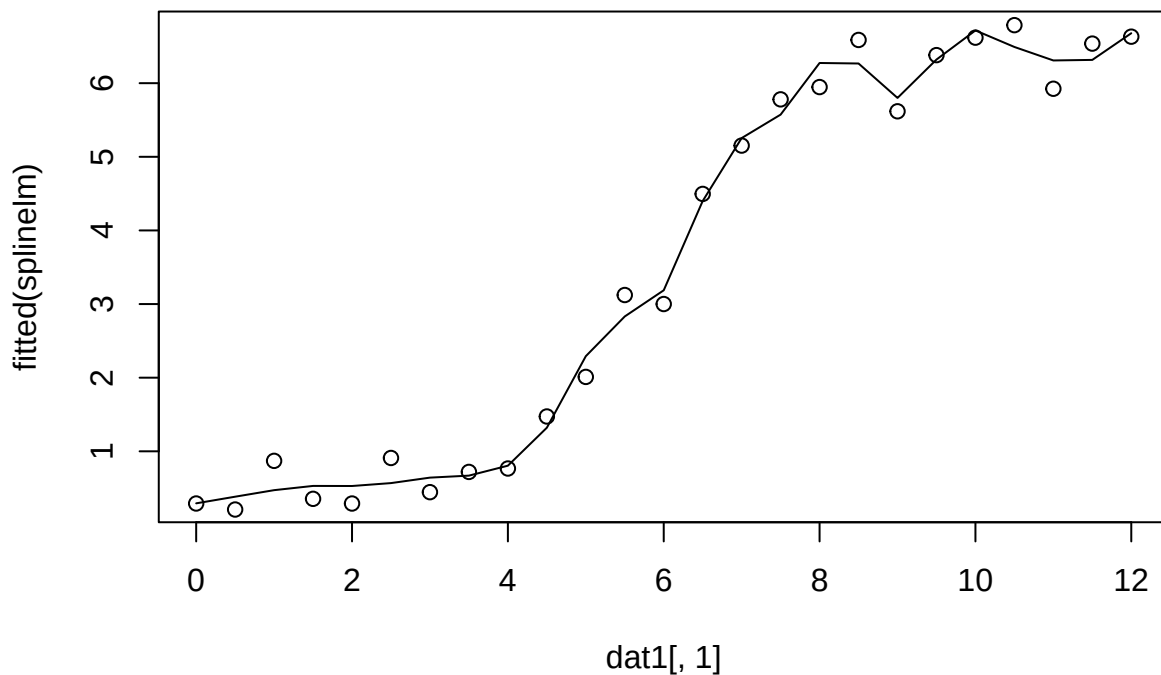
```
cubicspline(dat1,3)
```



```
## $'basis matrix'
##      [,1] [,2]      [,3]
## [1,] 1 0.0 0.0000
## [2,] 1 0.5 0.0000
## [3,] 1 1.0 0.0000
## [4,] 1 1.5 0.0000
## [5,] 1 2.0 0.0000
## [6,] 1 2.5 0.0000
## [7,] 1 3.0 0.0000
## [8,] 1 3.5 0.1250
## [9,] 1 4.0 1.0000
## [10,] 1 4.5 3.3750
## [11,] 1 5.0 8.0000
## [12,] 1 5.5 15.6250
## [13,] 1 6.0 27.0000
## [14,] 1 6.5 42.7500
## [15,] 1 7.0 63.0000
## [16,] 1 7.5 87.7500
## [17,] 1 8.0 117.0000
## [18,] 1 8.5 150.7500
## [19,] 1 9.0 189.0000
## [20,] 1 9.5 231.7708
## [21,] 1 10.0 279.1667
## [22,] 1 10.5 331.3125
```

```
## [23,] 1 11.0 388.3333
## [24,] 1 11.5 450.3542
## [25,] 1 12.0 517.5000
##
## $'linear model'
##
## Call:
## lm(formula = y ~ 0 + basismat)
##
## Coefficients:
## basismat1 basismat2 basismat3
## -0.889372  0.773253 -0.002351
```

```
cubicspline(dat1,15)
```



```
## $'basis matrix'
##      [,1] [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8]
## [1,] 1 0.0  0.000000  0.0000  0.000000  0.0000  0.000000  0.0000
## [2,] 1 0.5  0.000000  0.0000  0.000000  0.0000  0.000000  0.0000
## [3,] 1 1.0  0.015625  0.0000  0.000000  0.0000  0.000000  0.0000
## [4,] 1 1.5  0.421875  0.0000  0.000000  0.0000  0.000000  0.0000
## [5,] 1 2.0  1.953125  0.1250  0.000000  0.0000  0.000000  0.0000
## [6,] 1 2.5  5.359375  1.0000  0.015625  0.0000  0.000000  0.0000
## [7,] 1 3.0 11.390625  3.3750  0.421875  0.0000  0.000000  0.0000
## [8,] 1 3.5 20.796875  8.0000  1.953125  0.1250  0.000000  0.0000
## [9,] 1 4.0 34.328125 15.6250  5.359375  1.0000  0.015625  0.0000
## [10,] 1 4.5 52.734375 27.0000 11.390625  3.3750  0.421875  0.0000
## [11,] 1 5.0 76.765625 42.8750 20.796875  8.0000  1.953125  0.1250
## [12,] 1 5.5 107.171875 64.0000 34.328125 15.6250  5.359375  1.0000
## [13,] 1 6.0 144.703125 91.1250 52.734375 27.0000 11.390625  3.3750
## [14,] 1 6.5 190.109375 125.0000 76.765625 42.8750 20.796875  8.0000
```

```

## [15,] 1 7.0 244.140625 166.3750 107.171875 64.0000 34.328125 15.6250
## [16,] 1 7.5 307.546875 216.0000 144.703125 91.1250 52.734375 27.0000
## [17,] 1 8.0 381.078125 274.6250 190.109375 125.0000 76.765625 42.8750
## [18,] 1 8.5 465.484375 343.0000 244.140625 166.3750 107.171875 64.0000
## [19,] 1 9.0 561.515625 421.8750 307.546875 216.0000 144.703125 91.1250
## [20,] 1 9.5 669.921875 512.0000 381.078125 274.6250 190.109375 125.0000
## [21,] 1 10.0 791.453125 614.1250 465.484375 343.0000 244.140625 166.3750
## [22,] 1 10.5 926.859375 729.0000 561.515625 421.8750 307.546875 216.0000
## [23,] 1 11.0 1076.765625 857.2500 669.796875 511.8750 380.953125 274.5000
## [24,] 1 11.5 1241.316220 999.0192 790.472222 613.1439 464.503125 342.0185
## [25,] 1 12.0 1420.975446 1154.7692 924.000000 726.1364 558.646875 419.0000
##      [,9]      [,10]      [,11]      [,12]      [,13]      [,14]      [,15]
## [1,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [2,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [3,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [4,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [5,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [6,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [7,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [8,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [9,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [10,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [11,] 0.000000 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [12,] 0.015625 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [13,] 0.421875 0.0000 0.000000 0.00000 0.000000 0.00000 0.000000
## [14,] 1.953125 0.1250 0.000000 0.00000 0.000000 0.00000 0.000000
## [15,] 5.359375 1.0000 0.015625 0.00000 0.000000 0.00000 0.000000
## [16,] 11.390625 3.3750 0.421875 0.00000 0.000000 0.00000 0.000000
## [17,] 20.796875 8.0000 1.953125 0.12500 0.000000 0.00000 0.000000
## [18,] 34.328125 15.6250 5.359375 1.00000 0.015625 0.00000 0.000000
## [19,] 52.734375 27.0000 11.390625 3.37500 0.421875 0.00000 0.000000
## [20,] 76.765625 42.8750 20.796875 8.00000 1.953125 0.12500 0.000000
## [21,] 107.171875 64.0000 34.328125 15.62500 5.359375 1.00000 0.015625
## [22,] 144.703125 91.1250 52.734375 27.00000 11.390625 3.37500 0.421875
## [23,] 189.984375 124.8750 76.640625 42.75000 20.671875 7.87500 1.828125
## [24,] 243.158854 165.3929 106.189236 63.01667 33.343750 14.63889 4.369792
## [25,] 304.664062 213.1071 141.796875 88.20000 49.781250 24.00000 8.296875
##
## $'linear model'
##
## Call:
## lm(formula = y ~ 0 + basismat)
##
## Coefficients:
## basismat1 basismat2 basismat3 basismat4 basismat5 basismat6
## 0.29364 0.18022 -0.08086 0.26351 -0.37953 0.47999
## basismat7 basismat8 basismat9 basismat10 basismat11 basismat12
## 0.16163 -2.05053 3.87188 -4.77817 4.55108 -4.32199
## basismat13 basismat14 basismat15
## 5.22554 -5.40519 3.60943

```

It appears to function well.

Problem 4

Read in the data:

```
rm(list=ls())
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr    1.5.1
## v ggplot2    3.5.1      v tibble     3.2.1
## v lubridate  1.9.3      v tidyr      1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
dat1<-read_csv(".././../PollData/2024/HarrisTrumpPolls2024.csv")
```

```
## New names:
## Rows: 142 Columns: 7
## -- Column specification
## ----- Delimiter: "," chr
## (4): Pollster, Date, Sample, MOE dbl (2): Harris/Biden, Trump lgl (1): ...7
## i Use 'spec()' to retrieve the full column specification for this data. i
## Specify the column types or set 'show_col_types = FALSE' to quiet this message.
## * ' -> '...7'
```

Extract the dates on which the poll was taken and convert to R date format.

startdate will be a vector containing the start dates of each poll **enddate** will be a vector containing the end dates of each poll

```
startdate<-NULL
enddate<-NULL
for(i in 1:nrow(dat1))
{
  startdate<-c(startdate, strsplit(dat1$Date[i], c("-"))[[1]][1])  #strsplit takes the date (a string
  enddate<-c(enddate, strsplit(dat1$Date[i], c("-"))[[1]][2])      #and the second part is the end date.
}
```

convert dates to days since beginning of year:

```
startdate<-strptime(startdate, format="%m/%d")$yday  #strptime converts between different data formats.
enddate<-strptime(enddate, format="%m/%d")$yday      #yday gives the day of the year (1-365)
```

Find the sample size for each poll. Sample size will be a vector giving the sample size for each poll. LVRV will be a vector specifying whether it is likely voters or registered voters.

```
samplesize<-NULL
LVRV<-NULL
for(i in 1:nrow(dat1))
```

```
{
samplesize<-c(samplesize, strsplit(dat1$Sample[i], " ")[[1]][1]) # we use strsplit to split out the LV (
LVRV<-c(LVRV, strsplit(dat1$Sample[i], " ")[[1]][2])

}
samplesize<-as.numeric(samplesize)
```

Warning: NAs introduced by coercion

Assemble the final data matrix:

```
finaldat<-cbind(dat1[,1],startdate,enddate,samplesize,LVRV,dat1[,4:6])
#finaldat<-finaldat#[finaldat[,5]=="LV",] #Reduce to likely voters only if desired (didn't do in this

finaldat=finaldat[!is.na(samplesize),]
finaldat=finaldat[1:(nrow(finaldat)-1),]
```

Calculate the total number of poll respondents favoring Harris and favoring Trump for each day of the year. i loop over the polls. For each poll, it takes the responses for each candidate and divides them evenly over the days of the poll. Doing this across all polls then adds up the responses for each candidate

The number of responses for each candidate is calculated as (sample size)*(percent for candidate)

```
Dvotes<-rep(0,365) #will be a vector containing the number of democratic candidate responses for each
Rvotes<-rep(0,365) #ditto for Republican candidate
neithervotes<-rep(0,365) #will be a vector containing the number of responses that were neither D nor
for(j in 1:nrow(finaldat)) #loop over polls
{
  days<-seq(from=finaldat[j,2],to=finaldat[j,3],by=1) #days over which the poll was conducted
  Dvotes[days]<-Dvotes[days]+finaldat[j,7]/100*finaldat[j,4]/length(days) #calculate the number of o
  Rvotes[days]<-Rvotes[days]+finaldat[j,8]/100*finaldat[j,4]/length(days) #same for Romney
  neithervotes[days]<-neithervotes[days]+(100-finaldat[j,7]-finaldat[j,8])/100*finaldat[j,4]/length(d
}
}
```

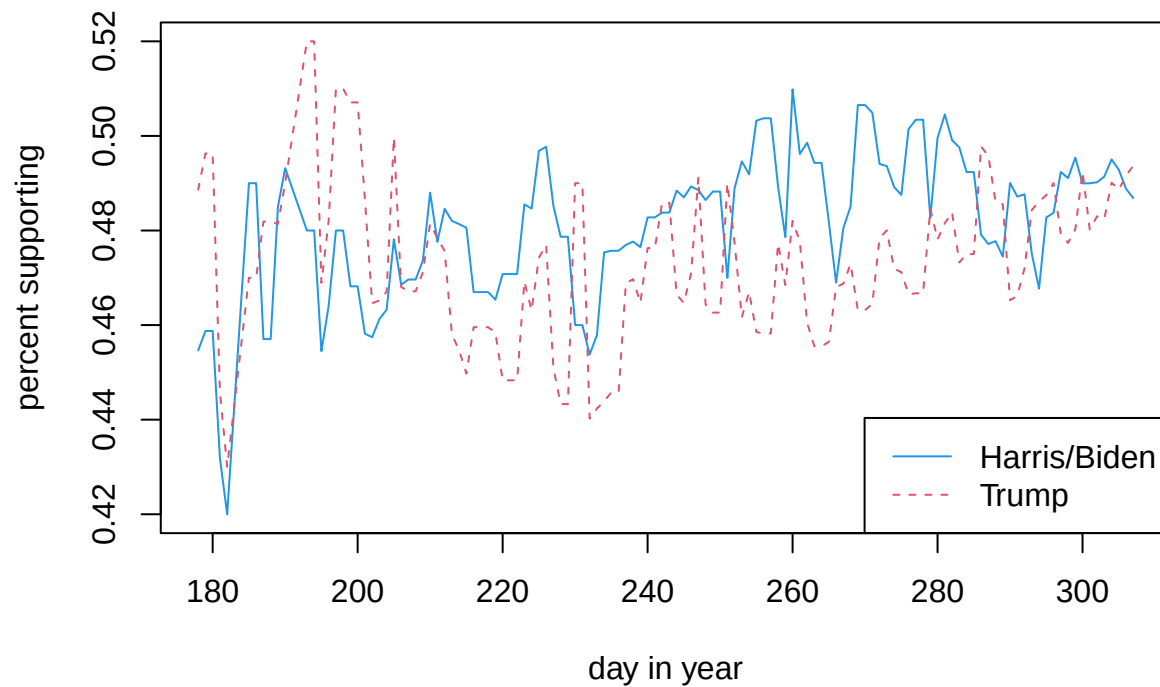
Calculate percentages supporting each candidate each day:

```
votes<-cbind(1:365,Rvotes,Dvotes,neithervotes) #matrix summarizing responses
votes<-votes[votes[,2]!=0,] #eliminate days with no polls covering them
percentD<-cbind(1:365,Dvotes/(Rvotes+Dvotes+neithervotes)) #find the percent of responses that were for
percentR<-cbind(1:365,Rvotes/(Rvotes+Dvotes+neithervotes)) #find the percent of responses that were for

percentD<-percentD[Dvotes!=0,] #eliminate days for which there were no polls.
percentR<-percentR[Dvotes!=0,]
```

This shows a line plots of the percents supporting each candidate each day:

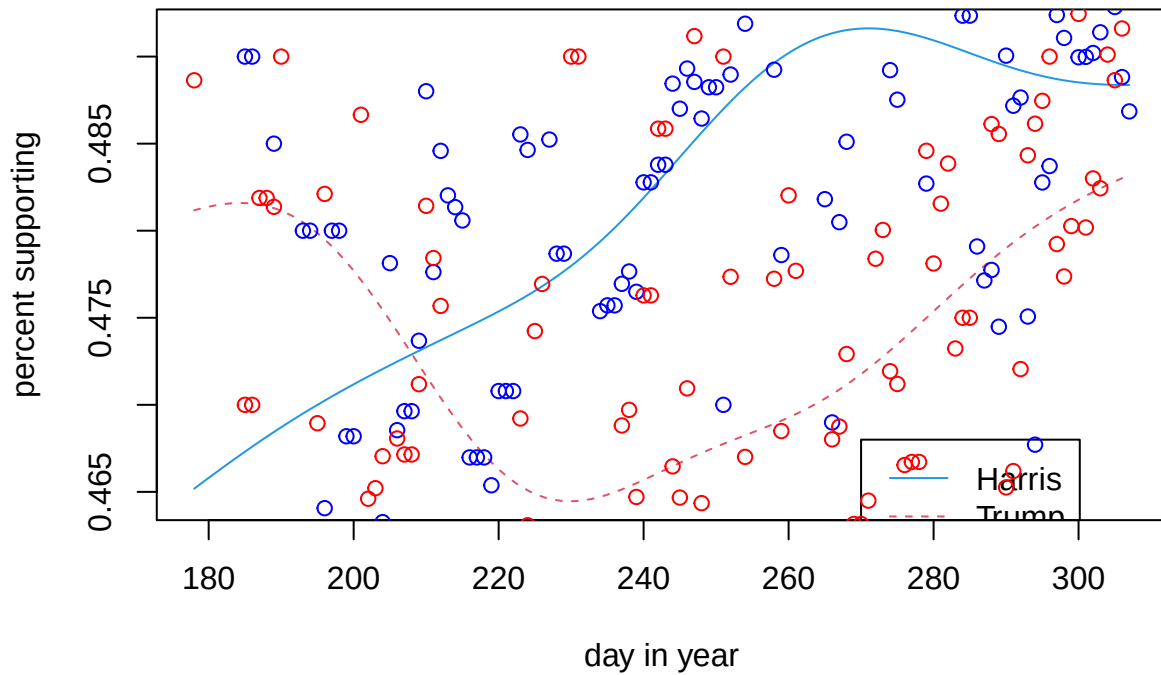
```
matplot(percentD[,1],cbind(percentD[,2],percentR[,2]),type="l", col=c(4,2),xlab="day in year", ylab="pe
legend("bottomright",c("Harris/Biden","Trump"),col=c(4,2),lty=1:2)
```



Plot kernel smoother with percents supporting each candidate.

```
#fit kernel smoother:
lambda<-40
PercentsOKS<-ksmooth(percentD[,1],percentD[,2],"normal",bandwidth=lambda)
PercentsRKs<-ksmooth(percentR[,1],percentR[,2],"normal",bandwidth=lambda)
dates<-PercentsOKS$x# strptime(PercentsOKS$x,format="%j")
matplot(dates,cbind(PercentsOKS$y,PercentsRKs$y),type="l",col=c(4,2),xlab="day in year", ylab="percent supporting")
legend(270,0.468,c("Harris","Trump"),col=c(4,2),lty=1:2)
points(percentD, col="blue")
points(percentR,col="red")
abline(v=312,lty=2)
```

percent supporting Harris and Trump in 2024 Polls



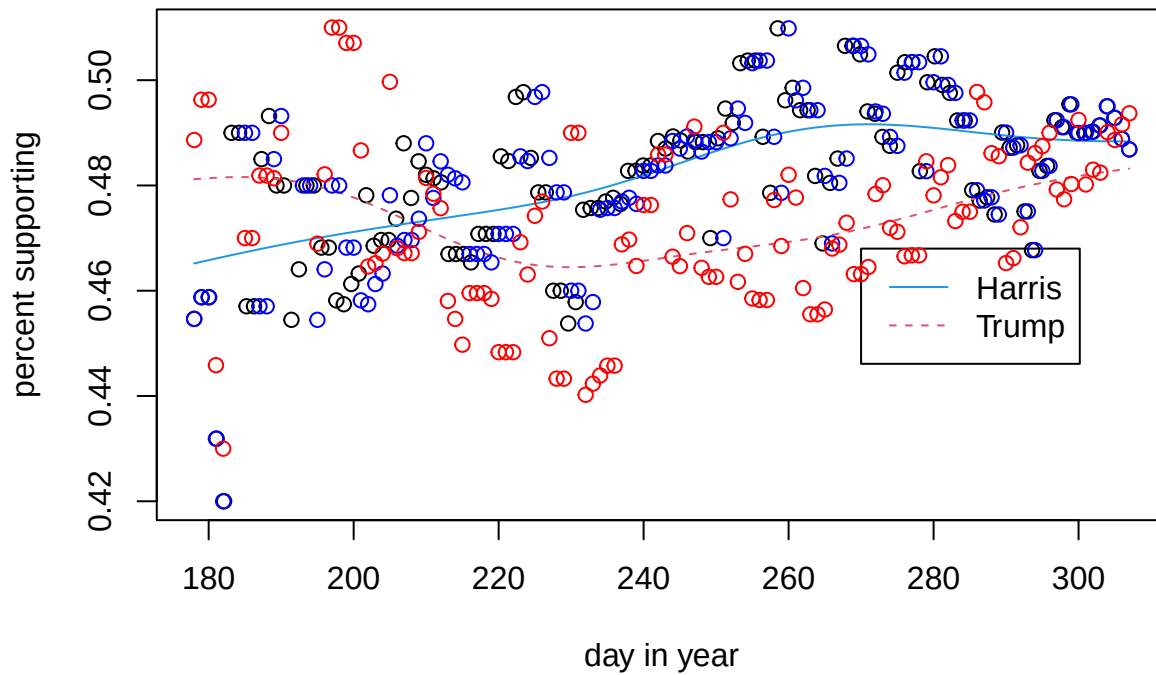
```
#plot(percentD[,1],percentD[,2])
```

This is the same plot, but with a larger y-axis range so that all points are included.

```
dates<-PercentsOKS$x# strptime(PercentsOKS$x,format="%j")
plot(dates,percentD[,2],xlab="day in year", ylab="percent supporting",main="percent supporting Harris and Trump",col=4)
lines(dates,PercentsOKS$y,col=4)
lines(dates,PercentsRKs$y,col=2,lty=2)
legend(270,0.468,c("Harris", "Trump"),col=c(4,2),lty=1:2)

points(percentD, col="blue")
points(percentR,col="red")
```


percent supporting Harris and Trump in 2024 Polls



The kernel smoother shows Harris with the lead at the end. This is of course not how thing turned out. It appears that the polls overestimated the support for Harris. Polling has become more difficult in recent years because of changes in how people communicate (in particular, the shift away from land lines). It may be that polls are not going to be accurate enough to predict a close race.